

Lab 04: Interface

Scenario 1: Digital Rights & Taxable Items

Concept: In a modern library, some items can be "Streamed Online," and some items are "Taxable" (like merchandise or books for sale). Since not all books are digital and not all library items are for sale, we use Interfaces to define these specific capabilities.

Instructions:

1. Reuse classes from Lab3 Scenario 1
2. Define Two Interfaces:
 - **DigitalContent:** Defines methods `streamOnline()` and `download()`.
 - **Taxable:** Defines the method `calculateTax()`.
3. Implement Interfaces in Subclasses:
 - **Class EBook:** Must implement both **DigitalContent** (can be read online) and **Taxable** ().
 - **Class PhysicalBook:** Does not implement **DigitalContent** but must implement **Taxable** ().
4. The System Logic:
 - In your Main class, create a method named `processDigitalAccess(DigitalContent item)`.
 - Show that this method can accept an EBook object, but it cannot accept a PhysicalBook object because a physical book does not follow the DigitalContent contract.
 - Specify that:
 1. Default price for e-books is 99.0 bath
 2. 5% tax for digital items and 7% tax for physical items
5. Challenge:
 - **Interface vs. Inheritance Hierarchy:** add a **LibraryMovie** class to our system. A movie is not a 'Book,' so it shouldn't be inherited from a Book class. However, a movie can be streamed online just like an E-book. How does the DigitalContent interface allow us to treat both a Movie and an EBook the same way in our streaming player?

Add extra code in main() for challenge

```
// Create a list for DigitalContent items (can contain both EBooks and Movies!)
List<DigitalContent> digitalContent = new ArrayList<>();
// Add movies (NOTE: Movies do NOT inherit from LibraryItem!)
LibraryMovie movie1 = new LibraryMovie("The Matrix", "Lana Wachowski",
    "https://streaming.library.com/matrix.mp4", 136, 1999, "Sci-Fi", 199.0);
LibraryMovie movie2 = new LibraryMovie("Inception", "Christopher Nolan",
    "https://streaming.library.com/inception.mp4", 148, 2010, "Sci-Fi/Thriller", 249.0);
// Add them to the digital content list
digitalContent.add(movie1);
digitalContent.add(movie2);
// Also add ebooks to the digital content list
digitalContent.add(new EBook("Effective Java", "Joshua Bloch", "978-0134685991",
    "https://library.ebooks.com/effective-java.pdf", 5.2));
// Demonstrate the Universal Streaming Player
System.out.println("\n" + "=".repeat(60));
System.out.println("  UNIVERSAL STREAMING PLAYER (Polymorphism via Interfaces)");
System.out.println("=".repeat(60));
System.out.println("\nThis demonstrates interface-based polymorphism:");
System.out.println("A single player can handle BOTH movies and e-books!\n");
// Play through each digital content without caring about their actual type
for (DigitalContent content : digitalContent) {
    if (content instanceof LibraryMovie) {
        LibraryMovie movie = (LibraryMovie) content;
        System.out.println("  STREAMING PLAYER - Playing Movie: " + movie.getTitle());
    } else if (content instanceof EBook) {
        EBook book = (EBook) content;
        System.out.println("  STREAMING PLAYER - Reading E-Book: " + book.getTitle());
    }
    System.out.println("---");
    launchStreamingPlayer(content);
}
```

6. Submit the upgrade code and sample output

Main class

```
package com.library.lab04;

import java.util.ArrayList;
import java.util.List;

public class LibraryManagementApp {

    public static void main(String[] args) {
        System.out.println("\n" + "=".repeat(60));
        System.out.println("  LIBRARY MANAGEMENT SYSTEM - POLYMORPHISM DEMO");
        System.out.println("=".repeat(60));

        // Create a list of LibraryItem (demonstrates polymorphism)
        List<LibraryItem> items = new ArrayList<>();

        // Add Physical Books
        items.add(new PhysicalBook("Java Programming", "John Smith", "978-0134685991",
450.0, "A1-04"));
        items.add(new PhysicalBook("Clean Code", "Robert Martin", "978-0132350884",
520.0, "B2-15"));
        items.add(new PhysicalBook("Design Patterns", "Gang of Four", "978-0201633612",
680.0, "A3-22"));

        // Add E-Books
        items.add(new EBook("Effective Java", "Joshua Bloch", "978-0134685991",
            "https://library.ebooks.com/effective-java.pdf", 5.2));
        items.add(new EBook("Python Crash Course", "Eric Matthes", "978-1593279288",
            "https://library.ebooks.com/python-crash.pdf", 8.7));

        // Instantiate member object
        Member member1 = new Member("M001", "Somsak");
        Member member2 = new Member("M002", "Suda");

        // Display all items using polymorphism
        System.out.println("\n--- ALL LIBRARY ITEMS (Polymorphism Demo) ---");
        System.out.println("Calling printSummary() on each item in the list:");
        System.out.println();
        for (LibraryItem item : items) {
            item.printSummary();
        }
    }
}
```

```

// Demonstrate checkOut() method
System.out.println("\n--- TESTING CHECKOUT FUNCTIONALITY ---");
System.out.println("\nMember Somsak borrows Physical Book:");
items.get(0).checkOut(member1);

System.out.println("\nMember Suda borrows E-Book:");
items.get(4).checkOut(member2);

System.out.println("\nAttempting to checkout an already borrowed item:");
items.get(0).checkOut(member2);

// Display items after checkout using polymorphism
System.out.println("\n--- ITEMS STATUS AFTER CHECKOUT ---");
for (LibraryItem item : items) {
    item.printSummary();
}

// Demonstrate returnItem() method
System.out.println("\n--- TESTING RETURN FUNCTIONALITY ---");
System.out.println("\nReturning Physical Book:");
items.get(0).returnItem();

// Test borrowing limit
System.out.println("\n--- TESTING BORROW LIMIT ---");
items.get(0).checkOut(member1);
items.get(1).checkOut(member1);
items.get(2).checkOut(member1);
items.get(3).checkOut(member1); // Should be denied (limit reached)

// ===== METHOD OVERRIDING: Late Fee Demo =====
System.out.println("\n" + "=".repeat(60));
System.out.println("  METHOD OVERRIDING: LATE FEE CALCULATION DEMO");
System.out.println("=".repeat(60));

// Simulate late returns
int daysLate = 5;
System.out.println("\n--- Late Fee Calculation (" + daysLate + " days late) ---");
System.out.println("\nPhysical Books (5 Baht per day late fee) and E-Books (NO
late fees - files auto-expire):");
for (LibraryItem item : items) {
    double lateFee = item.calculateLateFee(daysLate);
    System.out.printf("  %s: %.2f Baht\n", item.getTitle(), lateFee);
}

```

```

// ===== INTERFACE IMPLEMENTATION: Digital Content & Taxable Demo
System.out.println("\n" + "=".repeat(60));
System.out.println("  INTERFACE IMPLEMENTATION: DIGITAL CONTENT & TAXABLE DEMO");
System.out.println("=".repeat(60));

// Demonstrate Digital Content Interface
System.out.println("\n--- DIGITAL CONTENT INTERFACE ---");
System.out.println("Processing Digital Content for EBooks (DigitalContent
interface):");
    System.out.println("Note: Only EBooks implement DigitalContent, PhysicalBooks do
NOT.\n");

// This works because EBook implements DigitalContent
for (LibraryItem item : items) {
    if (item instanceof DigitalContent) {
        DigitalContent digitalBook = (DigitalContent) item;
        System.out.println("Processing EBook: " + item.getTitle());
        launchStreamingPlayer(digitalBook);
    }
}
// Demonstrate Taxable Interface
System.out.println("\n--- TAXABLE INTERFACE ---");
System.out.println("Processing Tax Calculation (Taxable interface):");
System.out.println("Both EBooks and PhysicalBooks implement Taxable.\n");

System.out.println("Tax Calculation for Physical Books (7% tax) and E-Books (5%
digital tax):");
for (LibraryItem item : items) {
    Taxable taxableItem = (Taxable) item;
    double tax = taxableItem.calculateTax();
    System.out.printf("  %s: Price = %.2f Baht, Tax = %.2f Baht, Total = %.2f
Baht\n",
        item.getTitle(), item.getPrice(), tax, item.getPrice() + tax);
}
}

```

```

/**
 * Universal Streaming Player - Works with ANY DigitalContent object!
 * This method demonstrates polymorphism through interfaces.
 * It doesn't care if it's a Movie or EBook - both implement DigitalContent!
 *
 * @param content Any object that implements DigitalContent (Movie, EBook, etc.)
 */
public static void launchStreamingPlayer(DigitalContent content) {
    System.out.println("Connecting to streaming service...");
    System.out.println("Loading content...\n");
    content.streamOnline();
    System.out.println("User requests offline copy:");
    content.download();
}
}

```

Example output:

```

=====
LIBRARY MANAGEMENT SYSTEM - POLYMORPHISM DEMO
=====

--- ALL LIBRARY ITEMS (Polymorphism Demo) ---
Calling printSummary() on each item in the list:

PHYSICAL BOOK
- Title:           Java Programming
- Author:          John Smith
- ISBN:            978-0134685991
- Price:           450.0 Baht
- Shelf Location:  A1-04
- Status:          Available
- Return Due Date: N/A (Book is available)

PHYSICAL BOOK
- Title:           Clean Code
- Author:          Robert Martin
- ISBN:            978-0132350884
- Price:           520.0 Baht
- Shelf Location:  B2-15
- Status:          Available
- Return Due Date: N/A (Book is available)

PHYSICAL BOOK
- Title:           Design Patterns
- Author:          Gang of Four
- ISBN:            978-0201633612
- Price:           680.0 Baht
- Shelf Location:  A3-22
- Status:          Available

```

```

- Return Due Date:      N/A (Book is available)

- Title:                Effective Java
- ISBN:                978-0134685991
- Status:              Available
- Return Due Date:      N/A (Item is available)

- Title:                Python Crash Course
- ISBN:                978-1593279288
- Status:              Available
- Return Due Date:      N/A (Item is available)

--- TESTING CHECKOUT FUNCTIONALITY ---

Member Somsak borrows Physical Book:
  Item 'Java Programming' has been checked out successfully.
  Item 'Java Programming' has been borrowed by Somsak.
  Return Due Date: 2026-02-11

Member Suda borrows E-Book:
  Item 'Python Crash Course' has been checked out successfully.
  Item 'Python Crash Course' has been borrowed by Suda.
  Return Due Date: 2026-02-11

Attempting to checkout an already borrowed item:
  Error: Item 'Java Programming' is already borrowed and cannot be checked out again.

--- ITEMS STATUS AFTER CHECKOUT ---
PhysicalBook[Title='Java Programming', Location='A1-04', Status='Borrowed']
PhysicalBook[Title='Clean Code', Location='B2-15', Status='Available']
PhysicalBook[Title='Design Patterns', Location='A3-22', Status='Available']
EBook[Title='Effective Java', Size='5.20 MB', Status='Available']
EBook[Title='Python Crash Course', Size='8.70 MB', Status='Borrowed']

--- TESTING RETURN FUNCTIONALITY ---

Returning Physical Book:
  Item 'Java Programming' has been returned successfully.

--- TESTING BORROW LIMIT ---
  Item 'Java Programming' has been checked out successfully.
  Item 'Java Programming' has been borrowed by Somsak.
  Return Due Date: 2026-02-11
  Item 'Clean Code' has been checked out successfully.
  Item 'Clean Code' has been borrowed by Somsak.
  Return Due Date: 2026-02-11
  Item 'Design Patterns' has been checked out successfully.
  Item 'Design Patterns' has been borrowed by Somsak.
  Return Due Date: 2026-02-11
  Member Somsak has reached the borrow limit (3).
  Borrow request denied for member Somsak.

=====
METHOD OVERRIDING: LATE FEE CALCULATION DEMO
=====

```

```

--- Late Fee Calculation (5 days late) ---

Physical Books (5 Baht per day late fee) and E-Books (NO late fees - files auto-
expire):
  Java Programming: 25.00 Baht
  Clean Code: 25.00 Baht
  Design Patterns: 25.00 Baht
  Effective Java: 0.00 Baht
  Python Crash Course: 0.00 Baht

=====
INTERFACE IMPLEMENTATION: DIGITAL CONTENT & TAXABLE DEMO
=====

--- DIGITAL CONTENT INTERFACE ---
Processing Digital Content for EBooks (DigitalContent interface):
Note: Only EBooks implement DigitalContent, PhysicalBooks do NOT.

Processing EBook: Effective Java
Connecting to streaming service...
Loading content...

  Streaming 'Effective Java' from URL: https://library.ebooks.com/effective-java.pdf
  Starting online stream... connected!
  You can now read the book online without downloading.
User requests offline copy:
  Downloading 'Effective Java' from URL: https://library.ebooks.com/effective-java.pdf
  Downloading file... (5.20 MB)
  Download complete! File saved to your device.
  You can now read the book offline.
Processing EBook: Python Crash Course
Connecting to streaming service...
Loading content...

  Streaming 'Python Crash Course' from URL: https://library.ebooks.com/python-
crash.pdf
  Starting online stream... connected!
  You can now read the book online without downloading.
User requests offline copy:
  Downloading 'Python Crash Course' from URL: https://library.ebooks.com/python-
crash.pdf
  Downloading file... (8.70 MB)
  Download complete! File saved to your device.
  You can now read the book offline.

--- TAXABLE INTERFACE ---
Processing Tax Calculation (Taxable interface):
Both EBooks and PhysicalBooks implement Taxable.

Tax Calculation for Physical Books (7% tax) and E-Books (5% digital tax):
  Java Programming: Price = 450.00 Baht, Tax = 31.50 Baht, Total = 481.50 Baht
  Clean Code: Price = 520.00 Baht, Tax = 36.40 Baht, Total = 556.40 Baht
  Design Patterns: Price = 680.00 Baht, Tax = 47.60 Baht, Total = 727.60 Baht
  Effective Java: Price = 99.00 Baht, Tax = 4.95 Baht, Total = 103.95 Baht
  Python Crash Course: Price = 99.00 Baht, Tax = 4.95 Baht, Total = 103.95 Baht

```


Scenario 2: Special Abilities & Interactable Objects

Concept: In a game, characters aren't the only things that can be interacted with. A wooden crate can be "Destroyed," and a potion can be "Used." Since a crate is not a "Character," we cannot use Inheritance. Instead, we use Interfaces to define these shared behaviors.

Instructions:

1. Reuse classes from Lab3 Scenario 2
2. Define Two Interfaces:
 - **Destructible:** Defines methods `takeDamage(int amount)` and `isDestroyed()`.
 - **Consumable:** Defines the method `use(Character user)`.
3. Implement Across Different Hierarchies:
 - **Class Character:** Must implement **Destructible** (because heroes take damage and can die).
 - **Class WoodenBox:** Create a new class (not inheriting from Character) and implement **Destructible** (it has HP and can be broken).
 - **Class HealthPotion:** Create a new class and implement **Consumable**.
4. The Interaction Logic:
 - **Modify the `Warrior.attack()` method.** Instead of accepting a **Character**, change the parameter to accept any **Destructible** object: `public void attack(Destructible target)`
 - **Demonstrate that the Warrior can now attack both a Mage and a WoodenBox using the exact same method.**
5. Challenge:
 - **Cross-Hierarchy Interaction:** By using an interface as a parameter in the `attack(Destructible target)` method, our game engine becomes significantly more flexible. Create a new classes: 'Stone Walls' and 'Castle Gates', which the Warrior can interact with immediately. These new objects implement the **Destructible** contract; the Warrior can attack them without requiring any changes or recompilation of the original Warrior class.
6. Submit the upgrade code and sample output

Main class

```
package com.rpg.lab04;

import java.util.ArrayList;
import java.util.List;

public class RPGGameApp {

    public static void main(String[] args) {
        System.out.println("\n" + "=".repeat(70));
        System.out.println("  SCENARIO 4: INTERFACES - DESTRUCTIBLE & CONSUMABLE");
        System.out.println("=".repeat(70));

        // ===== CREATE WEAPONS =====
        System.out.println("\n[STEP 1] Creating Weapons...");
        Weapon swordWeapon = new Weapon("Excalibur", "Melee", 30, "Slash Attack");
        Weapon staffWeapon = new Weapon("Staff of Mysticism", "Magic", 20, "Spell Cast");
        Weapon bowWeapon = new Weapon("Bow of Elven Make", "Ranged", 18, "Precise Shot");

        // ===== CREATE CHARACTERS =====
        System.out.println("\n[STEP 2] Creating Characters...");
        Warrior warrior = new Warrior("Aragorn", 12, 2000, 25, 8, 15, swordWeapon);
        Mage mage = new Mage("Gandalf", 14, 1200, 20, 5, 150, staffWeapon);
        Archer archer = new Archer("Legolas", 13, 1400, 22, 6, 85, bowWeapon);

        // ===== CREATE DESTRUCTIBLE OBJECTS (NON-CHARACTER) =====
        System.out.println("\n[STEP 3] Creating Destructible Objects (Non-Character)...");
        WoodenBox treasureBox = new WoodenBox("Treasure Chest", 300, true, "1000 Gold Coins");
        WoodenBox crate = new WoodenBox("Supply Crate", 150, false, "Medicine & Supplies");

        // ===== CREATE CONSUMABLE ITEMS =====
        System.out.println("\n[STEP 4] Creating Consumable Items...");
        HealthPotion potionGreater = new HealthPotion("Greater Health Potion", 100, 3);
        HealthPotion potionMinor = new HealthPotion("Minor Health Potion", 10, 10);

        // ===== DISPLAY INITIAL STATUS =====
        System.out.println("\n[STEP 5] Initial Status:");
        System.out.println("\n--- CHARACTERS (Destructible) ---");
        warrior.displayCharacterDetails();
        mage.displayCharacterDetails();
        archer.displayCharacterDetails();

        System.out.println("\n--- DESTRUCTIBLE OBJECTS (Non-Character) ---");
        treasureBox.displayBoxDetails();
```

```

crate.displayBoxDetails();

System.out.println("\n--- CONSUMABLE ITEMS ---");
potionGreater.displayPotionDetails();
potionMinor.displayPotionDetails();

// ===== DEMONSTRATE INTERFACE POLYMORPHISM: DESTRUCTIBLE
// =====
System.out.println("\n" + "=".repeat(70));
System.out.println(" [STEP 6] INTERFACE POLYMORPHISM: Destructible Objects");
System.out.println("=".repeat(70));
System.out.println("\n💎 KEY CONCEPT: Warriors can attack BOTH Characters and Objects!");
System.out.println(" Both implement Destructible, but DON'T share inheritance.\n");

// Create a list of destructible items (both Characters and non-Characters!)
List<Destructible> battleTargets = new ArrayList<>();
battleTargets.add(mage); // Character is Destructible
battleTargets.add(warrior); // Another Character is Destructible
battleTargets.add(treasureBox); // WoodenBox is Destructible (NOT a Character!)
battleTargets.add(crate); // Another WoodenBox

System.out.println("--- Aragorn and Mage attacks each other (Character - Destructible) ---");
for (int i = 0; i < 5; i++) {
    warrior.attack(battleTargets.get(1));
    mage.attack(battleTargets.get(0));
}

System.out.println("\n--- Aragorn continues attacking the Treasure Chest ---");
while (!battleTargets.get(2).isDestroyed()) {
    warrior.attack(battleTargets.get(2));
}

System.out.println("\n--- Legolas attacks a Supply Crate (Object - Destructible) ---");
archer.attack(battleTargets.get(3));

// ===== SHOW STATUS AFTER ATTACKS =====
System.out.println("\n[STEP 7] Status After Combat:");
System.out.println("\n--- Mage Status ---");
mage.displayCharacterDetails();

System.out.println("\n--- Destructible Objects Status ---");
treasureBox.displayBoxDetails();
crate.displayBoxDetails();

```

```

// ===== DEMONSTRATE INTERFACE POLYMORPHISM: CONSUMABLE
// =====
System.out.println("\n" + "=".repeat(70));
System.out.println(" [STEP 8] INTERFACE POLYMORPHISM: Consumable Items");
System.out.println("=".repeat(70));
System.out.println("\n💎 KEY CONCEPT: Different consumable items, same 'use' method!\n");

// Create a list of consumable items
List<Consumable> inventory = new ArrayList<>();
inventory.add(potionGreater);
inventory.add(potionMinor);

System.out.println("--- Gandalf uses a Greater Health Potion ---");
inventory.get(0).use(mage);

System.out.println("\n--- Gandalf uses a Minor Health Potion ---");
inventory.get(1).use(mage);

// ===== OPEN DESTROYED BOX =====
System.out.println("\n[STEP 9] Breaking Open Boxes:");
System.out.println("\n--- Breaking open the Treasure Chest ---");
treasureBox.breakOpen();

System.out.println("\n--- Attempting to break the Supply Crate (still intact) ---");
crate.breakOpen();
}
}

```

Example output:

```

=====
SCENARIO 4: INTERFACES - DESTRUCTIBLE & CONSUMABLE
=====

[STEP 1] Creating Weapons...
[STEP 2] Creating Characters...
[STEP 3] Creating Destructible Objects (Non-Character)...
[STEP 4] Creating Consumable Items...
[STEP 5] Initial Status:

--- CHARACTERS (Destructible) ---

--- ARAGORN (WARRIOR) ---
Status: Active
Level: 12
Health Points: 2000/2000
Damage: 25
Defense: 8

```

```

Armor Value: 15
Weapon: Excalibur (Type: Melee, Damage: 30, Ability: Slash Attack)

--- GANDALF (MAGE) ---
Status: Active
Level: 14
Health Points: 1200/1200
Mana: 150/150
Damage: 20
Defense: 5
Weapon: Staff of Mysticism (Type: Magic, Damage: 20, Ability: Spell Cast)

--- LEGOLAS (ARCHER) ---
Status: Active
Level: 13
Health Points: 1400/1400
Damage: 22
Defense: 6
Accuracy: 85%
Range Advantage: Yes (30% bonus)
Weapon: Bow of Elven Make (Type: Ranged, Damage: 18, Ability: Precise Shot)

--- DESTRUCTIBLE OBJECTS (Non-Character) ---

--- TREASURE CHEST (WOODEN BOX) ---
Status: Intact
Health: 300/300
Locked: Yes 🔒
Contains: 1000 Gold Coins

--- SUPPLY CRATE (WOODEN BOX) ---
Status: Intact
Health: 150/150
Locked: No 🗝️
Contains: Medicine & Supplies

--- CONSUMABLE ITEMS ---

--- GREATER HEALTH POTION ---
Healing Power: 100 HP
Quantity: 3
Type: Consumable

--- MINOR HEALTH POTION ---
Healing Power: 10 HP
Quantity: 10
Type: Consumable

=====
[STEP 6] INTERFACE POLYMORPHISM: Destructible Objects
=====

💡 KEY CONCEPT: Warriors can attack BOTH Characters and Objects!
Both implement Destructible, but DON'T share inheritance.

```

--- Aragorn and Mage attacks each other (Character - Destructible) ---

✂ Aragorn (Warrior) performs a POWERFUL STRIKE on Aragorn!

Base Damage: 55 → Enhanced: 82 (1.5x multiplier)

🛡 Aragorn's Armor: 15 (reduces damage from 82 to 67)

🛡 Aragorn's Defense: 8 (reduces damage from 67 to 59)

💥 Actual Damage Taken: 59

❤ Aragorn's HP: 1941/2000

💡 Gandalf (Mage) casts MAGIC MISSILE at Gandalf!

Spell Damage: 50 (Base: 40 + Spell Bonus: 10)

Mana Used: 30 | Remaining: 120/150

🛡 Gandalf's Defense: 5 (reduces damage from 50 to 45)

💥 Actual Damage Taken: 45

❤ Gandalf's HP: 1155/1200

✂ Aragorn (Warrior) performs a POWERFUL STRIKE on Aragorn!

Base Damage: 55 → Enhanced: 82 (1.5x multiplier)

🛡 Aragorn's Armor: 15 (reduces damage from 82 to 67)

🛡 Aragorn's Defense: 8 (reduces damage from 67 to 59)

💥 Actual Damage Taken: 59

❤ Aragorn's HP: 1882/2000

💡 Gandalf (Mage) casts MAGIC MISSILE at Gandalf!

Spell Damage: 50 (Base: 40 + Spell Bonus: 10)

Mana Used: 30 | Remaining: 90/150

🛡 Gandalf's Defense: 5 (reduces damage from 50 to 45)

💥 Actual Damage Taken: 45

❤ Gandalf's HP: 1110/1200

✂ Aragorn (Warrior) performs a POWERFUL STRIKE on Aragorn!

Base Damage: 55 → Enhanced: 82 (1.5x multiplier)

🛡 Aragorn's Armor: 15 (reduces damage from 82 to 67)

🛡 Aragorn's Defense: 8 (reduces damage from 67 to 59)

💥 Actual Damage Taken: 59

❤ Aragorn's HP: 1823/2000

💡 Gandalf (Mage) casts MAGIC MISSILE at Gandalf!

Spell Damage: 50 (Base: 40 + Spell Bonus: 10)

Mana Used: 30 | Remaining: 60/150

🛡 Gandalf's Defense: 5 (reduces damage from 50 to 45)

💥 Actual Damage Taken: 45

❤ Gandalf's HP: 1065/1200

✂ Aragorn (Warrior) performs a POWERFUL STRIKE on Aragorn!

Base Damage: 55 → Enhanced: 82 (1.5x multiplier)

🛡 Aragorn's Armor: 15 (reduces damage from 82 to 67)

 Aragorn's Defense: 8 (reduces damage from 67 to 59)
 Actual Damage Taken: 59
 Aragorn's HP: 1764/2000

✦ Gandalf (Mage) casts MAGIC MISSILE at Gandalf!
Spell Damage: 50 (Base: 40 + Spell Bonus: 10)
Mana Used: 30 | Remaining: 30/150
 Gandalf's Defense: 5 (reduces damage from 50 to 45)
 Actual Damage Taken: 45
 Gandalf's HP: 1020/1200

✂ Aragorn (Warrior) performs a POWERFUL STRIKE on Aragorn!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
 Aragorn's Armor: 15 (reduces damage from 82 to 67)
 Aragorn's Defense: 8 (reduces damage from 67 to 59)
 Actual Damage Taken: 59
 Aragorn's HP: 1705/2000

✦ Gandalf (Mage) casts MAGIC MISSILE at Gandalf!
Spell Damage: 50 (Base: 40 + Spell Bonus: 10)
Mana Used: 30 | Remaining: 0/150
 Gandalf's Defense: 5 (reduces damage from 50 to 45)
 Actual Damage Taken: 45
 Gandalf's HP: 975/1200

--- Aragorn continues attacking the Treasure Chest ---

✂ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
 Treasure Chest takes 82 damage!
 Treasure Chest HP: 218/300

✂ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
 Treasure Chest takes 82 damage!
 Treasure Chest HP: 136/300

✂ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
 Treasure Chest takes 82 damage!
 Treasure Chest HP: 54/300

✂ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
 Treasure Chest takes 82 damage!
 Treasure Chest HP: 0/300 (DESTROYED!)

--- Legolas attacks a Supply Crate (Object - Destructible) ---

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

💣 Supply Crate takes 52 damage!

📦 Supply Crate HP: 98/150

[STEP 7] Status After Combat:

--- Mage Status ---

--- GANDALF (MAGE) ---

Status: Active

Level: 14

Health Points: 975/1200

Mana: 0/150

Damage: 20

Defense: 5

Weapon: Staff of Mysticism (Type: Magic, Damage: 20, Ability: Spell Cast)

--- Destructible Objects Status ---

--- TREASURE CHEST (WOODEN BOX) ---

Status: Destroyed

Health: 0/300

Locked: Yes 🔒

Contains: 1000 Gold Coins

--- SUPPLY CRATE (WOODEN BOX) ---

Status: Intact

Health: 98/150

Locked: No 🔓

Contains: Medicine & Supplies

=====
[STEP 8] INTERFACE POLYMORPHISM: Consumable Items
=====

💡 KEY CONCEPT: Different consumable items, same 'use' method!

--- Gandalf uses a Greater Health Potion ---

🧴 Gandalf drinks Greater Health Potion!

❤️ Health restored: 975 → 1075 (+100)

🧴 Greater Health Potion remaining: 2

--- Gandalf uses a Minor Health Potion ---

🧴 Gandalf drinks Minor Health Potion!

❤️ Health restored: 1075 → 1085 (+10)

🧴 Minor Health Potion remaining: 9

[STEP 9] Breaking Open Boxes:

--- Breaking open the Treasure Chest ---

🔒 The lock broke! Treasure Chest breaks open!

📦 Contents revealed: 1000 Gold Coins

--- Attempting to break the Supply Crate (still intact) ---

⚠️ Cannot break open Supply Crate - it's still intact!

Challenge code for main:

```
// ===== CROSS-HIERARCHY INTERACTION =====
System.out.println("\n" + "=".repeat(70));
System.out.println(" [STEP 10] CROSS-HIERARCHY INTERACTION: Stone Walls &
Castle Gates");
System.out.println("=".repeat(70));
System.out.println("\n💎 KEY CONCEPT: Warrior can attack NEW object types
WITHOUT recompilation!");
System.out.println(" StoneWall and CastleGate implement Destructible - NO
changes to Warrior!\n");

// Create new destructible structures
System.out.println("[Creating New Structures]");
StoneWall northWall = new StoneWall("North Wall", 500, "Outer Wall");
StoneWall eastWall = new StoneWall("East Wall", 400, "Inner Wall");
CastleGate mainGate = new CastleGate("Main Gate", 600, true, true);
CastleGate sideGate = new CastleGate("Side Gate", 300, false, false);

// Display initial status
System.out.println("\n--- Initial Structure Status ---");
northWall.displayWallDetails();
eastWall.displayWallDetails();
mainGate.displayGateDetails();
sideGate.displayGateDetails();

// Warrior attacks stone walls
System.out.println("\n--- Aragorn (Warrior) attacks the North Wall ---");
for (int i = 0; i < 8; i++) {
    warrior.attack(northWall);
    if (northWall.isDestroyed()) {
        break;
    }
}

// Archer attacks castle gate
System.out.println("\n--- Legolas (Archer) attacks the Main Gate
(Reinforced!) ---");
for (int i = 0; i < 10; i++) {
    archer.attack(mainGate);
    if (mainGate.isDestroyed()) {
        break;
    }
}

// Warrior attacks side gate
System.out.println("\n--- Aragorn (Warrior) attacks the Side Gate ---");
while (!sideGate.isDestroyed()) {
```

```

        warrior.attack(sideGate);
    }

    // Show final status
    System.out.println("\n--- Final Structure Status ---");
    northWall.displayWallDetails();
    mainGate.displayGateDetails();
    sideGate.displayGateDetails();

    // Demonstrate polymorphism with mixed destructibles
    System.out.println("\n" + "=".repeat(70));
    System.out.println("  [STEP 11] POLYMORPHIC LIST: Mixed Destructible Objects");
    System.out.println("=".repeat(70));
    System.out.println("\n💎 All objects in one list - Characters, Boxes, Walls, Gates!\n");

    List<Destructible> allDestructibles = new ArrayList<>();
    allDestructibles.add(eastWall);           // StoneWall
    allDestructibles.add(crate);              // WoodenBox
    allDestructibles.add(archer);            // Character

    System.out.println("--- Mage attacking various targets from polymorphic list ---");
    for (Destructible target : allDestructibles) {
        if (target instanceof StoneWall) {
            System.out.println("\n>> Target: " + ((StoneWall) target).getLocation());
        } else if (target instanceof WoodenBox) {
            System.out.println("\n>> Target: " + ((WoodenBox) target).getName());
        } else if (target instanceof Character) {
            System.out.println("\n>> Target: " + ((Character) target).getName());
        }
        mage.attack(target);
    }
}

```

Example output of challenge code:

```

=====
[STEP 10] CROSS-HIERARCHY INTERACTION: Stone Walls & Castle Gates
=====

💎 KEY CONCEPT: Warrior can attack NEW object types WITHOUT recompilation!
  StoneWall and CastleGate implement Destructible - NO changes to Warrior!

[Creating New Structures]

--- Initial Structure Status ---

--- NORTH WALL ---
  Type: Outer Wall
  Status: STANDING
  Durability: 500/500

--- EAST WALL ---

```

```
Type: Inner Wall
Status: STANDING
Durability: 400/400

--- MAIN GATE ---
Status: INTACT
Durability: 600/600
Locked: Yes
Reinforced: Yes

--- SIDE GATE ---
Status: INTACT
Durability: 300/300
Locked: No
Reinforced: No

--- Aragorn (Warrior) attacks the North Wall ---

✖ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
🧱 North Wall takes 82 damage! (Durability: 418/500)

✖ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
🧱 North Wall takes 82 damage! (Durability: 336/500)

✖ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
🧱 North Wall takes 82 damage! (Durability: 254/500)

✖ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
🧱 North Wall takes 82 damage! (Durability: 172/500)

✖ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
🧱 North Wall takes 82 damage! (Durability: 90/500)

✖ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
🧱 North Wall takes 82 damage! (Durability: 8/500)

✖ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!
Base Damage: 55 → Enhanced: 82 (1.5x multiplier)
🧱 North Wall takes 82 damage! (Durability: -74/500)
💣 North Wall has CRUMBLed to pieces!

--- Legolas (Archer) attacks the Main Gate (Reinforced!) ---

🏹 Legolas (Archer) shoots a precise arrow at Target!
Base Damage: 40 + Range Bonus: 12 = 52
```

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 574/600)

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 548/600)

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 522/600)

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 496/600)

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 470/600)

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 444/600)

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 418/600)

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 392/600)

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 366/600)

🏹 Legolas (Archer) shoots a precise arrow at Target!

Base Damage: 40 + Range Bonus: 12 = 52

Accuracy: 85% (HIT!)

🛡 Main Gate is REINFORCED! (reduces damage from 52 to 26)

📊 Main Gate takes 26 damage! (Durability: 340/600)

--- Aragorn (Warrior) attacks the Side Gate ---

⚔ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!

Base Damage: 55 → Enhanced: 82 (1.5x multiplier)

📊 Side Gate takes 82 damage! (Durability: 218/300)

⚔ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!

Base Damage: 55 → Enhanced: 82 (1.5x multiplier)

📊 Side Gate takes 82 damage! (Durability: 136/300)

⚔ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!

Base Damage: 55 → Enhanced: 82 (1.5x multiplier)

📊 Side Gate takes 82 damage! (Durability: 54/300)

⚔ Aragorn (Warrior) performs a POWERFUL STRIKE on Target!

Base Damage: 55 → Enhanced: 82 (1.5x multiplier)

📊 Side Gate takes 82 damage! (Durability: -28/300)

💣 Side Gate has been BREACHED! The gate COLLAPSES!

--- Final Structure Status ---

--- NORTH WALL ---

Type: Outer Wall

Status: DESTROYED

Durability: 0/500

--- MAIN GATE ---

Status: INTACT

Durability: 340/600

Locked: Yes

Reinforced: Yes

--- SIDE GATE ---

Status: BREACHED

Durability: 0/300

Locked: No

Reinforced: No

=====
[STEP 11] POLYMORPHIC LIST: Mixed Destructible Objects
=====

```
✧ All objects in one list - Characters, Boxes, Walls, Gates!

--- Mage attacking various targets from polymorphic list ---

>> Target: East Wall

✧ Gandalf attempts to cast Magic Missile, but has insufficient mana!
  Current Mana: 0/150 (Need: 30)

>> Target: Supply Crate

✧ Gandalf attempts to cast Magic Missile, but has insufficient mana!
  Current Mana: 0/150 (Need: 30)

>> Target: Legolas

✧ Gandalf attempts to cast Magic Missile, but has insufficient mana!
  Current Mana: 0/150 (Need: 30)
```